

Contents

1	Introduction	1
2	Related Work	2
3	Methodology	3
3.1	Overview	3
3.2	Layer-wise text representations . .	3
3.3	Constructing AI-generated text di- rections	3
3.4	Projection features	4
3.5	Detection head	4
4	Experiments	4
4.1	Experimental setup	4
4.2	Experimental results	5
4.3	Ablation studies	7
5	What do the steering vectors detect?	8
6	Conclusion	9
7	Limitations and Future Work	10
A	Potential Risks	13
B	Inference Latency and Compute Over- head	13
C	Ablation on the choice of LLM backbone	15
D	Interpretability Analysis	16
D.1	Per-layer contribution to the detector	16
D.2	Logit-lens decoding of the steering vectors	16
D.3	Stylistic-feature baseline	17
D.4	Cross-LM Logit-lens decoding of steering vectors	17
D.5	Per-token visualization	18
E	Results on the COLING-2025 MGT benchmark	18
E.1	Setup	19
E.2	Results	19
E.3	Ablation on steering vector con- struction	19
F	Results Tables	21
F.1	DetectRL	21
F.2	MIRAGE	21
F.3	Ablation	21
F.4	Cross-benchmark transfer: bidirec- tional MIRAGE $\leftrightarrow$ DetectRL . . .	21

F.5	Cross-benchmark transfer to PAD- Ben and RAID	21
-----	--	----

G	Additional comparison with Repre- Guard	27
----------	--	-----------

A Potential Risks

Like other AI-generated text detectors, SV-Detect could be misused in high-stakes settings such as academic misconduct accusations, moderation, or authorship disputes if its predictions are treated as definitive evidence rather than probabilistic signals. False positives may unfairly penalize human authors, while false negatives may allow machine-generated text to evade detection. We therefore view SV-Detect as a decision-support tool rather than a standalone adjudicator.

A second risk is adversarial adaptation: public knowledge of the detector may encourage authors or model providers to modify generation style or post-process outputs in ways that reduce detectability. Although our experiments show robustness across several shifts and editing settings, we do not claim robustness to all future laundering or obfuscation strategies.

Finally, broad deployment of AI-generated text detectors may have downstream social effects, including over-policing legitimate writing assistance or disadvantaging users whose writing style differs from the training distribution. For these reasons, such systems should be deployed with clear uncertainty communication, human oversight, and regular re-evaluation on new domains and attack settings.

B Inference Latency and Compute Overhead

All measurements were obtained on a single NVIDIA A100-PCIe GPU (40 GB) in float16. We benchmark inference on 64 texts from MIRAGE-DIG/GENERATE (mean length: 980 characters), truncated to 512 tokens. Each detector is warmed up for three runs before timing. We report per-text latency at batch size 1, throughput at batch size 16, total parameter count (frozen backbone plus detector head), and peak GPU memory. We will release code to reproduce all experiments upon acceptance.

SV-Detect adds almost no overhead beyond a single LM forward pass. SV-Detect runs in

Detector	Params	Latency (ms/text, $b=1$)	Throughput (texts/s, $b=16$)	Peak GPU (MB)
SV-Detect (ours)	2.65 B	25.71	74.3	8,951
Log-Likelihood	2.65 B	27.63	75.1	7,746
Fast-DetectGPT	2.65 B	49.79	37.2	10,703
RoBERTa-Base	0.12 B	7.21	1,295	327
RoBERTa-Large	0.36 B	11.40	714	783

Table 1: Inference cost comparison on an A100 40 GB (fp16, 512-token cap, MIRAGE-DIG texts). Parameter counts include the frozen backbone and the detector head. For SV-Detect, the detector-specific overhead is only a small logistic-regression head on top of the LM representations.

25.7 ms per text, essentially matching a bare Log-Likelihood scoring pass through the same GPT-Neo-2.7B backbone (27.6 ms). The small advantage in favor of SV-Detect is expected: Log-Likelihood requires a full 50,257-way log-softmax at every token, whereas SV-Detect only uses hidden states, followed by lightweight mean pooling, cosine projections, and a tiny logistic-regression head. In practice, the detector-specific computation is negligible compared to the backbone forward pass.

SV-Detect is much cheaper than perturbation-based LLM detectors. Fast-DetectGPT requires two forward passes through a 2.7B-class model and is therefore about $1.9\times$ slower than SV-Detect in our setup (49.8 ms vs. 25.7 ms per text). The gap would be even larger for DetectGPT, whose cost scales linearly with the number of perturbations. Using the standard $K=100$ setting, a simple extrapolation from the Log-Likelihood baseline yields roughly 2.8 s per text, i.e. around $100\times$ the cost of SV-Detect. We therefore do not benchmark DetectGPT separately, since its overhead is determined directly by repeated LM forward passes.

RoBERTa-based supervised detectors are cheaper, but less robust. RoBERTa-Base and RoBERTa-Large are substantially faster than SV-Detect, with latencies of 7.2 ms and 11.4 ms per text, respectively. The throughput difference is even larger at batch size 16, reflecting both smaller parameter counts and lower memory pressure. This is the main compute-robustness trade-off of our approach: SV-Detect inherits the cost of an LLM forward pass, whereas small encoder-based detectors are far cheaper. However, as shown in the main experiments, these lighter supervised baselines degrade substantially under source-model and attack-family shifts. In contrast, SV-Detect remains close to the cost of a single

zero-shot LM forward while providing much stronger generalization.

Training overhead is modest. The training cost of SV-Detect is dominated by a single offline pass through the reference LM to extract layer-wise activations, which can then be cached once per benchmark. The downstream classifier operates on only L or $L \times 3$ features and fits in seconds on a CPU. By contrast, RoBERTa-based supervised baselines require full encoder fine-tuning for each training setup, which is the main difference in training cost rather than inference cost.

C Ablation on the choice of LLM backbone

In the main experiments, SV-Detect uses GPT-Neo as the probe LLM in order to match the backbone used by prior work and enable a fair comparison. Here, we study how the method changes when the probe backbone is replaced with a different small open-weight LLM. We consider four alternatives with parameter counts broadly comparable to GPT-Neo: Qwen/Qwen3-1.7B, Qwen/Qwen3-1.7B-Base, google/gemma-3-1b-pt, and google/gemma-3-1b-it. This also lets us compare pretrained/base and instruction-tuned variants within the same model family.

Table 2 reports AUROC on DetectRL Multi-Domain transfer. Overall, all tested backbones perform strongly, with near-perfect diagonal performance and robust off-diagonal transfer. The strongest results are obtained with the Qwen backbones. Both Qwen3-1.7B and Qwen3-1.7B-Base improve over GPT-Neo on most train–test pairs, and Qwen3-1.7B-Base is the strongest overall. In particular, it gives the best off-diagonal transfer when trained on ARXIV, XSUM, WRITING, or REVIEW, suggesting that its representation geometry is especially well suited to learning stable human-vs-AI directions.

GPT-Neo remains competitive, but is generally weaker than the Qwen variants on the harder cross-domain pairs, especially those involving transfer into or out of WRITING. For example, the ARXIV $\rightarrow$ WRITING and XSUM $\rightarrow$ WRITING pairs improve substantially under both Qwen backbones.

The Gemma backbones are also competitive. Gemma3-1B-pt performs strongly across most transfer pairs and often approaches GPT-Neo, while Gemma3-1B-it achieves similarly high diagonal performance and remains robust under transfer. Overall, Gemma backbones remain slightly less consistent than Qwen on the hardest off-diagonal pairs, but still yield strong AUROC throughout.

Taken together, these results indicate that SV-Detect is not tied to GPT-Neo and remains effective across several backbone families. Backbone choice still matters, however: Qwen-style representations provide the strongest and most stable transfer, while GPT-Neo and Gemma remain competitive but slightly less robust on the hardest cross-domain pairs.

GPT-Neo-2.7B				
Train	ArX.	XSum	Writ.	Rev.
ArXiv	100.00	97.34	86.82	94.05
XSum	97.80	99.94	83.02	95.24
Writing	86.74	81.82	99.98	99.40
Review	94.17	93.96	99.38	99.96
Qwen3-1.7B				
Train	ArX.	XSum	Writ.	Rev.
ArXiv	100.00	99.34	87.49	96.28
XSum	99.40	99.98	96.29	98.84
Writing	91.23	91.69	99.98	99.60
Review	97.24	98.43	99.48	99.93
Qwen3-1.7B-Base				
Train	ArX.	XSum	Writ.	Rev.
ArXiv	100.00	99.24	94.21	98.19
XSum	99.50	99.98	97.39	99.41
Writing	95.29	94.02	99.99	99.76
Review	97.87	97.94	99.77	99.96
Gemma3-1B-pt				
Train	ArX.	XSum	Writ.	Rev.
ArXiv	100.00	98.26	89.97	96.51
XSum	98.77	99.98	94.96	98.48
Writing	86.19	93.72	99.98	99.67
Review	93.72	96.41	99.32	99.97
Gemma3-1B-it				
Train	ArX.	XSum	Writ.	Rev.
ArXiv	100.00	98.83	86.38	96.05
XSum	98.43	99.94	95.13	98.16
Writing	85.76	95.58	99.98	99.30
Review	95.44	96.60	99.03	99.94

Table 2: Ablation on the probe LLM backbone for DetectRL Multi-Domain transfer, per-cell AUROC (rows = training domain, columns = evaluation domain).